

Modernizing Data Processing: CNNs and OCR for Automated Document Classification and Data Extraction

Soham Patel

Department of Computer Engineering
NMIMS, Mukesh Patel School of Technology
Management and Engineering
Mumbai, India
soham.patel63@nmims.edu.in

Dhyey Sanghavi

Department of Computer Engineering
NMIMS, Mukesh Patel School of Technology
Management and Engineering
Mumbai, India
dhyey.sanghavi76@nmims.edu.in

Prof. Archana Nanade

Department of Computer Engineering
NMIMS, Mukesh Patel School of Technology
Management and Engineering
Mumbai, India
archana.nanade@nmims.edu

Abstract— The growing use of identification documents such as Aadhar cards in the digital age has underlined the significance of creating simplified methods for automated classification and data extraction. In this paper, we present a comprehensive exploration and practical implementation of cutting-edge technologies aimed at revolutionizing data analysis, information retrieval, and automation, particularly in the context of identifying documents such as Aadhar cards in the digital age. Our approach integrates Convolutional Neural Networks (CNNs) for classification and the Tesseract OCR engine for data extraction, demonstrating their combined potential for driving significant advancements. We systematically tested and optimized various architectural elements, including different layer configurations, dropout rates, batch normalization placements, and regularizers. Through meticulous experimentation, we achieved remarkable results and substantial progress across multiple dimensions. Notably, our final model, strategically leveraging a hybrid optimization strategy involving SGD, RMSprop, and Adam optimizers, along with architectural refinements, attained an impressive accuracy of 97.52%. This accomplishment underscores the effectiveness of our approach in automated classification and data extraction tasks.

Keywords— *image classification; data extraction; Convolution Neural Networks (CNN); Tesseract-OCR; Adam; RMSprop; SGD*

I. INTRODUCTION

In the contemporary digital era, the widespread utilization of identification documents, exemplified by Aadhar cards, has underscored the urgency for the development of streamlined methods for automated classification and data extraction. The current state-of-the-art system for document processing and data privacy entails significant time investment and necessitates improvements to expedite and optimize the entire workflow. In the age of rapid technological advancement, the fusion of machine learning, image classification, Convolutional Neural Networks (CNN), and data extraction through the Tesseract Optical Character Recognition (OCR) engine is driving a paradigm shift in various domains. This paper presents a comprehensive exploration and practical implementation of these cutting-edge technologies, showcasing their combined potential to revolutionize data analysis, information retrieval, and automation.

Machine learning has emerged as a transformative force, capable of unraveling complex patterns and making sense of vast datasets. By enabling computers to learn from experience, machine learning algorithms have been employed to solve diverse problems across industries. In this study, our focus converges on image classification, where machine learning techniques are leveraged to categorize and interpret visual data, thus automating the process of identifying objects, scenes, or entities. In the realm of image classification, Convolutional Neural Networks (CNNs) have emerged as the linchpin of progress. Their architecture, inspired by the visual processing mechanisms of the human brain, exhibits unparalleled aptitude in capturing intricate features from images. This paper not only provides a comprehensive understanding of CNNs but also delves into their practical implementation. By utilizing real-world datasets and fine-tuning pre-trained models, we demonstrate how CNNs can be tailored to specific tasks, thereby exemplifying their role in solving complex image classification challenges.

While image classification [1] excels in visual understanding, data extraction from images—particularly textual information—remains a critical endeavor. The Tesseract OCR engine, fortified with the capabilities of Long Short-Term Memory (LSTM) networks, offers a robust solution for transforming images into actionable text [11][12]. Our study extends beyond theoretical exploration by showcasing the integration of Tesseract OCR with LSTM in practical scenarios. By navigating through intricate layouts and nuanced fonts, we illustrate how this technology combination can be harnessed to extract meaningful textual data from diverse image sources. This research paper aspires not only to elucidate the theoretical underpinnings of machine learning, image classification, CNNs, and Tesseract OCR but also to present their tangible implementation across domains. We aim to showcase how these technologies, when seamlessly integrated, can transcend conceptual boundaries and bring about tangible enhancements. By presenting practical use cases and methodologies, we underscore the practicality of these tools in solving real-world challenges.

II. PROBLEM STATEMENT AND SOLUTION

In the context of contemporary digital operations, the utilization of identification documents, such as Aadhar cards,

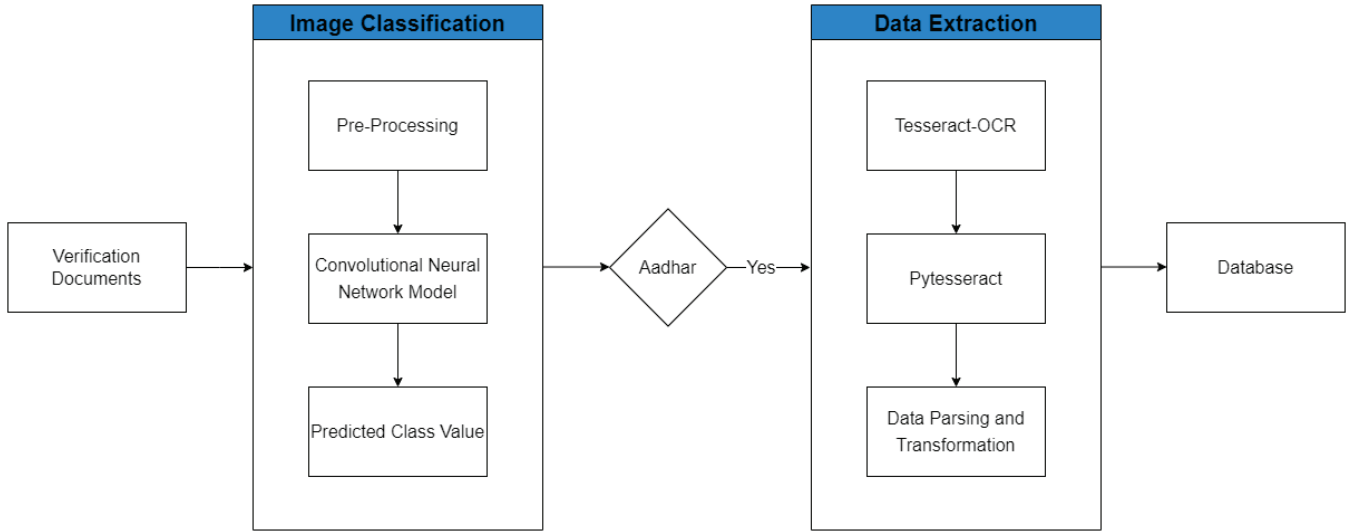


Fig. 1. Solution Framework

has accentuated the necessity for efficient methodologies in automated classification and data extraction. Presently, the prevailing verification and masking system is entangled in a laborious process, involving exhaustive scanning of entire PDF documents submitted for verification. This process endeavors to locate personal authentication documents harboring sensitive numerical data, aided by an Application Programming Interface (API) that applies masking techniques to preserve individual privacy. However, inherent within this approach are formidable challenges that necessitate resolution to enhance efficacy and operational efficiency.

The most glaring predicament lies in the inefficient utilization of computational resources, where each PDF, irrespective of containing the sought-after identification document, undergoes comprehensive scanning. Consequently, this surplus scanning incurs a substantial time overhead, hampering the system's responsiveness. Moreover, the reliance on third-party data extraction services introduces financial burdens and delays, as each document subjected to external processing incurs costs and elongates processing timelines. The indiscriminate scanning process heightens privacy and compliance concerns, as sensitive data is processed unnecessarily, risking inadvertent exposure and regulatory non-compliance.

The existing verification process also lacks targeted data extraction, leading to inefficiencies in verification and additional form filling. The inability to harness data from authentication documents further limits the system's potential to enhance verification procedures and streamline data management. To surmount these challenges, our proposed solution advocates a novel approach to document classification and processing, integrating advanced machine learning techniques to enhance system efficiency, accuracy, and overall performance. The cornerstone of our solution lies in the incorporation of a Convolutional Neural Network (CNN)-based classifier into the existing workflow.

The envisioned workflow commences with a preliminary scan of the entire PDF, accomplishing dual objectives: detecting document presence and optimizing data extraction. Should the document of interest be identified within the PDF, the system seamlessly advances to the CNN-based classifier stage. Through a series of convolutional operations and neural network inference, the document type is accurately ascertained. This innovative approach affords efficient and precise masking, wherein only classified identification documents necessitate processing through the masking API. This targeted approach eradicates the need to transmit entire PDF files, thus drastically reducing processing time and optimizing overall system performance.

Furthermore, our solution extends its benefits to data extraction, alleviating dependence on third-party services. By integrating the Tesseract OCR engine, classified document images undergo autonomous data extraction, facilitating streamlined form filling and data aggregation. This in-house data extraction not only curtails operational costs but also accelerates the document processing workflow.

In summation, the challenges posed by the existing verification and masking system necessitate a transformative solution. By integrating a CNN-based classifier and Tesseract OCR with LSTM, our proposed approach enhances classification accuracy, accelerates processing, and introduces targeted data extraction. This comprehensive solution addresses inefficiencies, reduces costs, and augments the verification process, culminating in a paradigm shift in automated document management and data security.

III. METHODOLOGY

In alignment with our solution's framework, as illustrated in Figure 1, the framework accommodates a diverse range of document files, expanding beyond sole reliance on images. This adaptability reflects the real-world complexity of document processing. The model's underlying concept remains centered on document classification, with a nuanced workflow tailored to effectively handle various types of files.

At the model's inception, it assimilates a collection of document files in a spectrum of formats such as images, PDFs, and other relevant file types, acknowledging the multifaceted nature of document submission. The pivotal stage involves the initial classification of these document files into discrete categories: those containing "adhar" information and those without. This categorization serves as the foundation for subsequent processing, streamlining the path for efficient data extraction and masking.

Once classification is completed, the classified document files undergo a text extraction module. This module systematically extracts relevant textual data from the files, irrespective of the format. The incorporation of the Tesseract OCR engine, complemented by LSTM networks, empowers the model to adeptly discern and extract necessary sensitive information. Within this extracted data lies the information pertinent to "aadhar" documents. These extracted "aadhar" data are subsequently subjected to privacy masking techniques, bolstered by the Application Programming Interface (API), ensuring compliance with privacy regulations and safeguarding sensitive data.

The extracted "aadhar" data, securely masked for privacy, are meticulously stored within a dedicated database. This repository serves as a central hub for organized data storage, fostering accessibility and expediting subsequent processes. The manifold applications of the extracted data, ranging from seamless verification procedures to streamlined form filling and data aggregation, underline the model's versatility and practical utility.

A. Image Classification

Image classification [1] anchors our solution, reshaping document processing. Powered by advanced Convolutional Neural Networks (CNNs), our model swiftly identifies "aadhar" credentials in document images, catalyzing subsequent workflow. This image processing prowess ensures precision, forming a robust foundation. The strategic integration of image processing fortifies privacy measures, ensuring secure data handling and regulatory compliance. Beyond document categorization, image processing holds broader significance. It converts intricate visual data into actionable insights, fostering streamlined operations and enriched user experiences. Our solution, fueled by image processing's transformative capabilities, redefines document management, enhancing efficiency, accuracy, and security.

1) Data Preprocessing

In the realm of data preprocessing, we use operations aimed at preparing PDF document pages for subsequent analysis and classification. It's noteworthy to emphasize that the nature of our input data predominantly consists of authentication documents, a characteristic that inherently minimizes the need for extensive data cleaning measures.

The process commences with the initiation of a PDF document through the utilization of the 'fitz' library. This library empowers the extraction of pages, and the subsequent iteration through each page paves the way for individual page processing. As part of this processing, the content of each page is loaded and translated into a pixmap representation, which

serves as the intermediary format for subsequent transformations. One of the pivotal aspects of data preprocessing involves the conversion of the pixmap data into a standard image format, a step enabled by the 'PIL' (Python Imaging Library) module. This translation facilitates seamless compatibility with various image processing techniques and lays the groundwork for further analysis.

To ensure uniformity and mitigate computational complexity, a standardized image size of 256x256 pixels is achieved through image resizing. This standardization is critical in maintaining consistency in the input data presented to the subsequent classification phase, a fundamental building block of our solution. The transition from image to array representation transpires through TensorFlow's 'image.img_to_array()' function. This transformation facilitates the translation of the image's visual content into a numerical format, making it amenable for integration into machine learning models. A quintessential step in the data preprocessing pipeline; pixel normalization is performed. By dividing each pixel value by 255.0, the pixel values are confined to the range of [0, 1]. This normalization process not only aligns the pixel values for improved model convergence but also enhances the stability of the training process.

Moreover, it is prudent to underscore that while our preprocessing strategy acknowledges the potential presence of blurred images, the application of image sharpening techniques is intentionally deferred until after the classification phase. This strategic decision is rooted in the insight that sharpening images post-classification can substantially refine the extraction of textual data from classified "aadhar" documents. Given the predominant cleanliness and classification of our input documents, this selective utilization of image sharpening optimizes the efficiency and precision of subsequent classification and information extraction tasks.

In conclusion, the intricate operations encapsulated within the provided code snippet illuminate the meticulous steps undertaken to transform raw PDF document pages into a format that seamlessly aligns with the requirements of image classification. By aligning with the inherent cleanliness of classified documents and applying image sharpening selectively, our data preprocessing approach stands as a testament to our overarching goal of optimizing efficiency, accuracy, and precision throughout the solution's workflow.

2) Convolutional Neural Network Model

Convolutional Neural Networks (CNNs) represent a class of deep learning models designed specifically for image analysis and processing tasks. They automatically learn hierarchical features from input images, making them effective in tasks such as image classification, object detection, and segmentation. CNNs use convolutional layers for localized feature extraction [2], followed by pooling layers to down sample features and reduce complexity [2]. This architecture enables learning intricate patterns, improving accuracy and efficiency in image-based tasks.

Our model is the result of iterative fine-tuning through rigorous experimentation and optimization. Meticulous

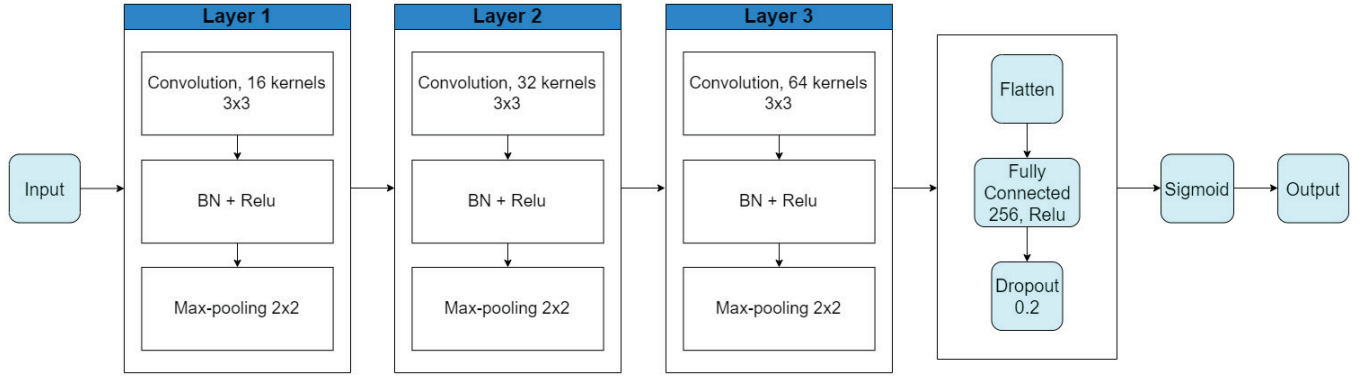


Fig. 2. CNN Architecture and Working

adjustments to architecture and hyperparameters created a model that maximizes accuracy and efficiency, seamlessly integrating with the data preprocessing steps previously outlined.

During the fine-tuning process, we tackled the challenge of overfitting, a common issue in deep learning. This problem arose as we experimented with varying filter and layer numbers. An overly complex model resulted in overfitting, where the network memorized data rather than learning generalized features. To counteract this, we strategically used regularization techniques like L1 or L2 regularization to control complexity and prevent overfitting. Our iterative refinement included careful consideration of dropout layers to mitigate reliance on specific neurons, enhancing generalization. Batch normalization promoted convergence and stability during training, mitigating overfitting. We meticulously evaluated and selected optimizers tailored to our use case, guiding weight updates during training for optimal convergence and reduced overfitting risk.

The model is structured sequentially, stacking layers to process input images. The foundational layer is a Convolutional layer ('Conv2D') with 16 filters of size 3x3. These filters are activated using the Rectified Linear Unit (ReLU) function to capture nonlinear patterns. Input dimensions are configured as (256, 256, 3) for 256x256-pixel RGB images. This layer includes L2 regularization for complexity management and generalization. A subsequent Batch Normalization layer enhances training stability, and a MaxPooling2D layer downsamples feature maps to reduce complexity. This pattern continues through two more convolutional layers, each with 32 and 64 filters of size 3x3. ReLU activation, Batch Normalization, and MaxPooling2D contribute to feature extraction and spatial downscaling.

Following the convolutional layers, a Flatten layer transforms the multidimensional feature maps into a one-dimensional vector, facilitating seamless compatibility with fully connected (Dense) layers. Two Dense layers follow suit. The first of these layers comprises 256 neurons activated by ReLU, with an added Dropout layer set at a rate of 0.2. This strategic use of Dropout helps counteract overfitting tendencies by inhibiting over-reliance on specific neurons. The final Dense layer, composed of a single neuron, operates through the sigmoid activation function, aptly suited for

binary classification tasks such as our "aadhar" document detection.

The model optimization process entails employing Stochastic Gradient Descent (SGD) as the chosen optimizer, fine-tuned with a learning rate of 0.001. The model's compilation encompasses binary cross-entropy loss, well-suited for binary classification endeavors. Additionally, the model tracks the accuracy metric during training, providing insights into its performance. The training phase involves iterating over the training dataset ('train') for 100 epochs. To monitor the model's progress and avoid overfitting, a validation dataset ('validation') is incorporated.

With our refined Convolutional Neural Network (CNN) model proficient in distinguishing between "adhar" and "not adhar" documents, we now transition to the subsequent crucial phase: data extraction. Beyond the realm of classification, this step holds significant importance as it involves extracting essential textual information from the identified documents.

B. Data Extraction

Data extraction stands as a pivotal process in diverse domains, where information retrieval from various sources takes center stage. Utilizing advanced techniques, data extraction involves the retrieval of meaningful insights from unstructured or structured data. This process extends beyond mere classification, focusing on the seamless retrieval of specific details, such as text, numbers, or patterns, from diverse formats like documents, images, or web pages. With applications spanning finance, healthcare, research, and automation, data extraction is a crucial element in transforming raw data into actionable insights, thereby enhancing decision-making, efficiency, and knowledge acquisition.

1) Tesseract OCR

In our project, after successfully classifying "adhar" documents using our Convolutional Neural Network (CNN) model, the focus shifts seamlessly to the crucial task of data extraction. This phase involves systematically retrieving relevant textual information embedded within the classified documents.

Data extraction is facilitated by integrating Tesseract OCR [7], a cutting-edge Optical Character Recognition engine.

Tesseract OCR transforms visual data from classified images into actionable text [11], bridging the gap between image-based content and computable information. The extraction process involves several steps. [7] Classified document images undergo Tesseract OCR's algorithms, systematically analyzing visual components. Tesseract OCR identifies textual patterns regardless of fonts, styles, or orientations, culminating in meaningful data extraction from images. Extracted textual information holds multifaceted utility. Beyond its role in classification verification, it aids streamlined form filling and data aggregation. This structured data enriches databases, enhancing data-driven insights and informed decision-making.

The harmonious interplay between image classification and data extraction is central to our solution, boosting efficiency and expanding its scope. This fusion results in a robust document processing solution that accurately classifies documents and unlocks the potential of data extraction, reshaping information management and utilization.

2) *Extraction Process with Pytesseract*

Our project logic encapsulates a powerful automation process for extracting essential information from images of Aadhar cards. This process revolves around Optical Character Recognition (OCR), a technology that converts printed or handwritten text from images into machine-readable text. At the heart of this functionality is the Tesseract OCR library, a well-established and widely used tool for OCR tasks. The Python function named 'all_data_extraction' serves as the central orchestrator of this process. It employs Tesseract, along with other libraries such as OpenCV and regular expressions, to meticulously extract critical details from Aadhar card images.

The process begins by loading an image using the OpenCV library's 'cv2.imread' function. The image is loaded in grayscale mode ('0' argument) to simplify subsequent processing. Next, the Tesseract library's 'image to string' function is employed to extract the textual content present in the image. These details include the Aadhar number, name, date of birth (DOB), gender, and address of the cardholder.

3) *Data Transformation and Parsing*

In the subsequent stages, the function initiates data transformation, and parsing procedures. Initially, the Aadhar card image is transformed and processed using OpenCV to enhance its clarity. Tesseract then comes into play, converting the processed image into text data. Regular expressions are employed to isolate and extract specific patterns, such as the Aadhar number and date of birth. The function meticulously parses the extracted data, fine-tuning it to eliminate unwanted characters or elements.

To facilitate the extraction of specific details, the code defines several nested utility functions:

1. 'extract_aadhaar_number': This function utilizes a regular expression pattern to identify and extract the 12-digit Aadhar number. The Aadhar number typically follows a specific format of four digits, a space, four digits, another space, and finally, four more digits.

2. 'extract_aadhaar_name': The goal of this function is to extract the name of the Aadhar cardholder. It does so by employing a series of regular expression patterns, accommodating various name formats that might consist of one, two, or three parts (first name, middle name, last name).

3. 'extract_aadhaar_dob': Here, the focus is on extracting the date of birth (DOB) from the extracted text. Regular expressions are employed to recognize patterns such as "dd/mm/yyyy." Additionally, the function is designed to identify keywords like "yob" (year of birth) or "birth" to capture the DOB.

4. 'extract_aadhaar_address': This function is responsible for extracting the address details from the text. It employs regular expressions to capture address-related information, even if the address spans multiple lines or is rotated.

5. To determine the gender of the cardholder, the code searches for gender-related keywords such as "Male," "MALE," "Female," and "FEMALE" within the extracted text.

In scenarios where OCR might fail due to incorrect text orientation, the code includes a mechanism to address this issue. It applies a 90-degree counterclockwise rotation to the image using the 'cv2.rotate' function. This rotated image is then subjected to text extraction attempts for the name, Aadhar number, and address. Extracted data undergoes a data cleaning process. This includes addressing formatting issues in the address, such as handling line breaks, multiple commas, equal signs, and certain symbols like replacing "!" with "l."

By combining OCR, regular expressions, and image rotation, the code ensures reliable extraction even in situations where text orientation, layout, and formatting may vary. In essence, the integration of these functions offers a robust and efficient solution for automated data extraction, empowering streamlined management, and analysis of Aadhar card information. This not only enhances operational efficiency but also serves as a foundation for various applications that rely on accurate and structured Aadhar data.

IV. RESULTS AND INFERENCE

Through meticulous experimentation and iterative refinement, our team has developed a series of CNN models to address a specific task – accurately classifying and processing images. In this section, we delve into the outcomes of these models, each representing a distinct phase of improvement and adaptation. From the initial model to the final deployment on the server, the journey of enhancing accuracy, optimizing architectures, and incorporating additional data showcases the evolution and efficacy of our CNN models.

Initial Dataset: The initial model was constructed with an image size of 256 and comprised three convolutional and max pooling layers. The filter sequence used was 16, 32, and 16. Despite its simplicity, this model achieved an accuracy of 48.26%, setting a baseline for further enhancements.

1st Training Iteration: In the first iteration, the model's architecture remained consistent with the initial design. However, early stopping based on validation accuracy was introduced to mitigate overfitting. This refinement

significantly improved the accuracy to 87.53%, showcasing the benefits of early stopping.

2nd Training Iteration: The second iteration aimed to expand the model's complexity by altering the filter sequence to 32, 64, and 128, while retaining the 256-node dense hidden layer. The result was an accuracy of 86.43%, indicating that deeper architectures could lead to performance gains.

3rd Training Iteration: The third iteration circled back to the original architecture [4], omitting batch normalization and dropout. Surprisingly, this approach led to a notable accuracy boost to 96.38%. This outcome underscored the intricate balance between regularization techniques and model architecture.

4th Training Iteration: In the fourth iteration, the model introduced batch normalization and dropout with a rate of 0.4. An additional dense hidden layer with 128 nodes was incorporated. This ensemble approach produced an accuracy of 89.26%.

5th Training Iteration: With a larger image size of 512, the fifth iteration retained the ensemble configuration with batch normalization, dropout (0.3), and the added hidden layer. This adjustment led to an accuracy of 89.87%, showcasing the significance of input size on model performance.

6th Training Iteration: Returning to the original image size of 256, the sixth iteration-maintained batch normalization and dropout (0.2). Moreover, an L2 regularizer was introduced for the dense hidden layer. This combination resulted in an accuracy of 96.2% and a loss of 0.2175, emphasizing the impact of regularization on model robustness.

Final Model (7th Training - SGD): The ultimate configuration involved filters of 16, 32, and 64. The model incorporated batch normalization, dropout (0.2), and an L2 regularizer, while introducing a new convolutional layer with more filters. This final model achieved the highest accuracy of 97.52% and validation loss 0.1342, showcasing the culmination of architectural finesse and regularization techniques.

The iterative progression of these models exemplifies the intricate decision-making process involved in designing and fine-tuning convolutional neural networks, showing how each adjustment contributes to overall model performance and accuracy. The selection of the appropriate optimizer plays a pivotal role in enhancing Convolutional Neural Networks (CNNs) training.

Adam optimizer [8] combines Adaptive Moment Estimation (Adam) and Root Mean Square Propagation (RMSprop) to adapt learning rates for each parameter, accelerating convergence and achieving remarkable results. It balances step size by considering historical gradients and squared values, ensuring robustness to varying learning rates.

RMSprop addresses the diminishing learning rate issue of stochastic gradient descent (SGD) by maintaining a moving average of squared gradients and adjusting learning rates based on recent gradient history [9]. This prevents fluctuations and oscillations in learning rates, leading to faster convergence and stability.

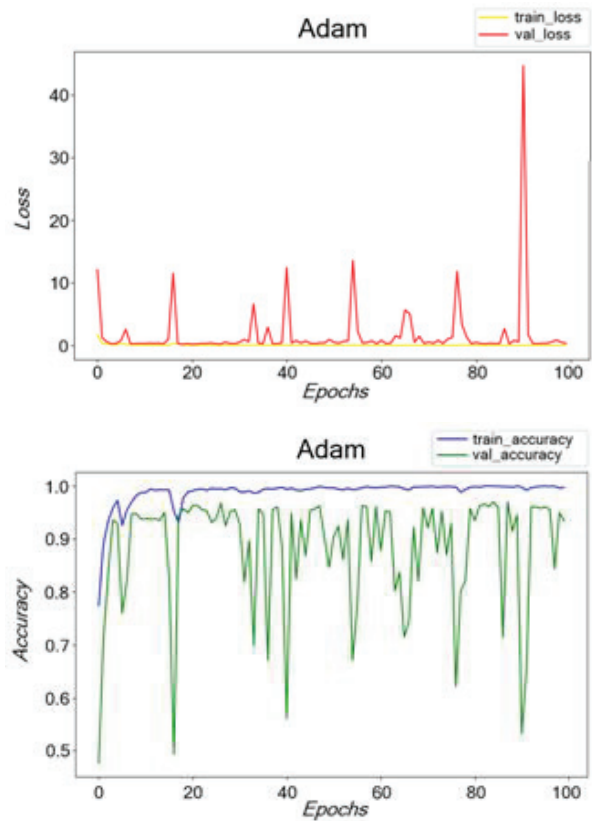


Fig. 3. Accuracy and Loss for optimizer Adam

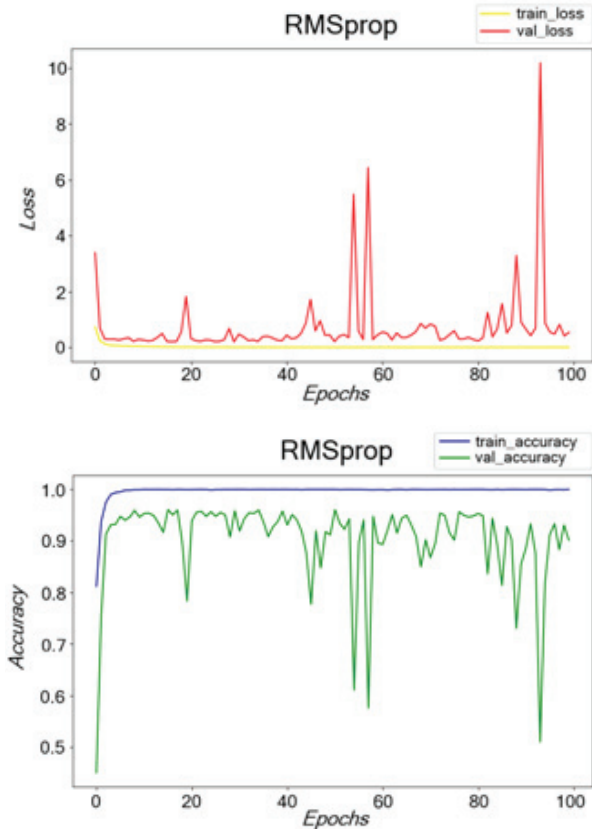


Fig. 4. Accuracy and Loss for optimizer RMSprop

SGD, the classic optimization algorithm, updates model parameters in the opposite direction of the gradient to minimize the loss function [10]. While simple and powerful, it can have slow convergence, especially in complex networks. The superior performance of SGD optimizer in our CNN model is attributed to its controlled learning rate updates. Unlike Adam and RMSprop, which adapt learning rates for each parameter based on historical gradients, SGD maintains a more direct and controlled approach.

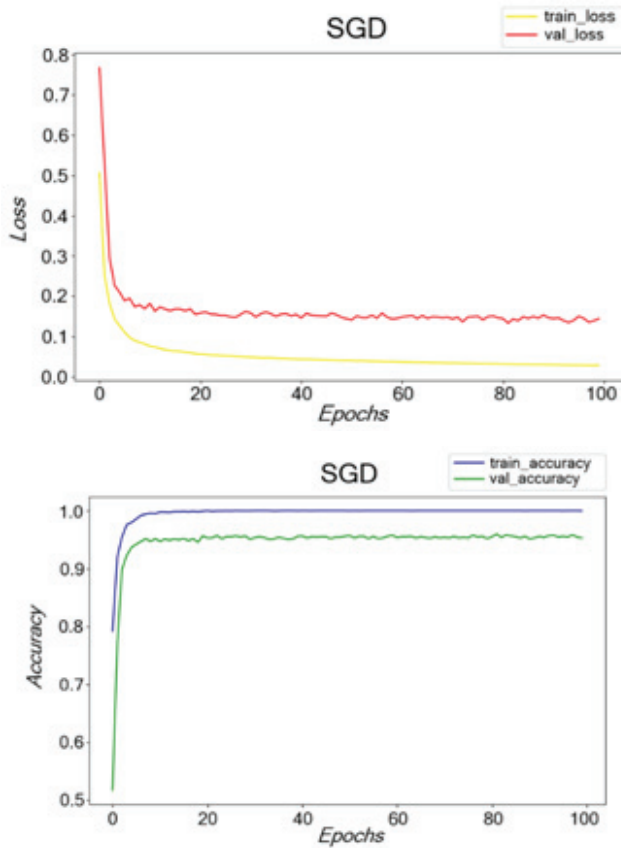


Fig. 5. Accuracy and Loss for optimizer SGD

During training, the network updates its parameters in the direction opposite to the gradient to minimize the loss function [6]. The controlled nature of SGD's learning rate adjustments prevents rapid fluctuations seen with other optimizers [10]. This controlled approach often allows the model to converge to more optimal points in the loss landscape, leading to improved convergence. By utilizing SGD in our final iteration, we adopted a stable and controlled learning rate approach. This enabled the model to navigate the optimization landscape more effectively, resulting in improved accuracy. The switch to SGD at this stage likely stemmed from the dataset's characteristics, model architecture, and the need for controlled fine-tuning towards training's end.

In some scenarios, Adam and RMSprop optimizers [8][9], while offering quick initial convergence, can sometimes lead to overshooting or erratic movement during later training stages. This can cause the model to miss finer loss landscape details and settle for suboptimal solutions.

In conclusion, the journey of enhancing the CNN model unveiled the intricacies of designing robust architectures for image classification. The iterative improvements highlighted the synergy between model architecture, regularization, and optimizer selection. This approach culminated in a remarkable accuracy of 97.52%, validating the meticulous decision-making process and showcasing the power of adaptive learning and regularization techniques in crafting highly accurate CNN models.

The seamless transition between CNN's classification and subsequent data extraction underscores the holistic nature of our approach. Achieving classification accuracy rates as high as 97.52%, our CNN model not only provided precise classifications but also laid the foundation for accurate data extraction. This phase involves extracting critical personal information, such as names, genders, dates of birth, Aadhar numbers, and addresses, from the classified documents. This integration of classification and extraction processes ensures a cohesive workflow that significantly minimizes both time and effort.

The implementation of our model significantly reduces processing time. Previously, classifying and masking Aadhar documents took around 11 minutes, while the new approach achieves classification and extraction within 5-6 seconds. Additionally, the financial cost and effort associated with data extraction are markedly reduced due to in-house processing, nearly eliminating the need for external per-document charged services.

File Path	File Name	Aadhar	Name	Gender	DOB	Aadhar Number	Address
XXXX	92 (2).jpg	Front side	BrXXXXXX	Male	1/1/1999	XXXX XXXX 4734	S/O VXXXX PaXXXX, A/XX, Xth Floor, ShXXXX VinXXXX BuXXXX X, TejXXXX SchXXXX XXX XX,X, VXXX PaXXXX (EXXXX), MXXXX, MaXXXX XXXX, 40XXXX
XXXX	aadhaar_back_180.png	Back side				XXXX XXXX 8695	
XXXX	aadhaar_front_270.jpg	Front side	SoXXXX VXXXX PaXXX	Male	9/12/2002	XXXX XXXX 8695	
XXXX	IMG-4979_180.jpg	Front side	JXXXX RaXXXX MeXXX	Male	15/01/2002	XXXX XXXX 4774	

Fig. 6. Database storing extracted data from classified images

These outcomes demonstrate our model's efficacy in accurately extracting personal information from diverse Aadhar documents, as represented in figure 6. The model identifies key details like Aadhar numbers, names, genders, dates of birth, and addresses. This implementation underscores the model's practicality and efficiency in processing substantial document volumes while ensuring accuracy and security. The data in figure 6 has been masked for privacy reasons.

In conclusion, the transition from CNN classification to data extraction highlights our model's comprehensive utility. It excels not only in accurate classification but also in expediting data extraction. This integration harnesses the strengths of each phase, crafting a robust tool for efficient and precise Aadhar document processing.

V. CONCLUSIONS

Our project stands as a significant achievement in document classification and data extraction, specifically focusing on Aadhar cards. Our aim was to enhance accuracy, efficiency, and privacy in handling sensitive documents. By merging Convolutional Neural Networks (CNNs) for classification and the Tesseract OCR engine for data extraction, we achieved remarkable results and substantial progress across various dimensions.

Through meticulous experimentation and iterative refinement, accuracy saw consistent improvement across iterations. Additional data dumps and architectural enhancements, including batch normalization, dropout, and L2 regularization, played pivotal roles in gradually enhancing accuracy. Our final model, utilizing a hybrid optimization strategy of SGD, RMSprop, and Adam, alongside architectural adjustments, attained an impressive 97.52% accuracy—a notable accomplishment demonstrating the effectiveness of our approach. Moreover, the exponential reduction in processing time further magnifies our achievement. Previously, document classification and masking took around 11 minutes; now, it's done in a mere 5-6 seconds. This substantial time reduction underscores the transformative impact on processing efficiency. Additionally, our solution led to substantial cost and effort savings. The data extraction cost, previously 2 Rs per document, is nearly eliminated by our in-house Tesseract OCR-powered solution. This economic benefit, combined with time savings, positions our project as an operational efficiency catalyst.

In the future, our project lays the foundation for document processing, OCR, and sensitive data extraction advancements. It highlights AI-driven automation's potential, coupled with

OCR technology, to revolutionize document management, achieving faster, cost-effective, and accurate outcomes. Our experience underscores AI technology's potential when effectively deployed, propelling us towards a future marked by intelligent, efficient, and secure data handling.

REFERENCES

- [1] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems (NIPS)*.
- [2] Shi, B., Bai, X., & Yao, C. (2017). An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- [3] Smith, J. S. (2017). On the Origin of Deep Learning. *arXiv preprint arXiv:1702.07800*.
- [4] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [6] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [7] Tesseract OCR - GitHub Repository. <https://github.com/tesseract-ocr/tesseract>
- [8] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.
- [9] Tieleman, T., & Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*.
- [10] Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010* (pp. 177-186). Physica-Verlag HD.
- [11] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [12] Greff, K., Srivastava, R. K., Koutnik, J., Steunebrink, B. R., & Schmidhuber, J. (2016). LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*.